

ARM CPU 엣지 디바이스를 위한 INT8 양자화 YOLOv8의 후처리 최적화: 양자화 인식 보정, 크기 인식 임계값, 캐시 인식 선택적 디코딩

김성빈¹ · 박현철^{1,*}

¹국립한국교통대학교

Post-Processing Optimization of INT8-Quantized YOLOv8 for ARM CPU Edge Devices: Quantization-Aware Calibration, Size-Aware Thresholding, and Cache-Aware Selective Decoding

Sung-been Kim¹ · Hyun-chul Park^{1,*}

¹Korea National University of Transportation

E-mail : rlatjdqls0509@ut.ac.kr / hc.park@ut.ac.kr

요 약

INT8로 양자화된 검출기를 ARM CPU에 배포하면 좌표 회귀가 훼손되어 소형 객체 정확도가 떨어지고, 모든 앵커를 디코딩하는 표준 후처리는 메모리 측면에서 비효율적이다. 본 논문은 추론이 전체 지연시간의 99% 이상을 차지해 후처리 최적화가 처리량에 영향을 주지 않는다는 점을 이용하여, 후처리에서 정확도와 메모리 효율을 함께 높이는 세 기법(양자화 인식 보정, 크기 인식 신뢰도 임계값, 캐시 인식 선택적 디코딩)과 그 통합을 제안한다. Raspberry Pi 4에서 YOLOv8n으로 평가한 결과, 통합 기법은 기준선과 같은 16.3 FPS에서 mAP@0.5를 0.6203에서 0.6407로 높였고, 선택적 디코딩은 검출 결과를 바꾸지 않고 후처리 지연시간을 65% 단축하였다.

ABSTRACT

Deploying INT8-quantized detectors on ARM CPUs degrades coordinate regression, which lowers small-object accuracy, while the standard post-processing that decodes every anchor is memory-inefficient. Since inference takes over 99% of end-to-end latency, changes in post-processing cannot affect throughput. We exploit this headroom and propose three post-processing techniques, namely quantization-aware calibration, a size-aware confidence threshold, and cache-aware selective decoding, together with their integration. On a Raspberry Pi 4 with YOLOv8n, the integrated method keeps the baseline's 16.3 FPS while raising mAP@0.5 from 0.6203 to 0.6407, and selective decoding cuts post-processing latency by 65% without changing any detection.

키워드

INT8 Quantization, Object Detection, Post-processing, Selective Decoding, Edge Inference

I. 서 론

* 교신저자

치매 환자 돌봄용 홈 카메라는 배회·낙상과 같은 위험 상황을 수 초 안에 감지해야 하고, 사생활 보호를 위해 영상을 외부로 보내지 않는 온디바이스 추론이 필요하며, 보급을 위해 저가·저전력

ARM CPU에서 동작해야 한다. 이를 위한 표준적 접근은 경량 검출기를 INT8 정수 연산으로 양자화하여 ARM CPU에 배포하는 것이다 [1]. 그러나 검출기 전체를 양자화하면 특히 좌표 회귀가 양자화에 취약해 성능이 크게 떨어진다는 [2]. 또한 모든 앵커를 단일 신뢰도 임계값으로 디코딩하는 표준 후처리는 소형 객체를 놓치기 쉽고, 대부분 폐기될 앵커의 좌표까지 읽어 메모리 접근이 비효율적이다.

본 연구는 대상 보드에서 추론이 전체 지연시간의 99% 이상을 차지한다는 관찰에서 출발한다. 후처리를 최적화해도 처리량은 변하지 않지만, 바로 그 때문에 후처리는 처리량 손실 없이 정확도와 메모리 효율을 높일 수 있는 여유 공간이 된다. 이에 본 논문은 양자화 인식 보정(B), 크기 인식 신뢰도 임계값(C), 캐시 인식 선택적 디코딩(D)과 이를 결합한 통합 기법(E)을 제안한다.

II. 관련 연구

양자화 연구는 회귀 특화 양자화 [2]처럼 보정 손실과 양자화기 설계를 바꾸어 좌표 품질을 개선하며, 디코딩 직전의 INT8 좌표를 보정하지는 않는다. 후처리 가속 연구는 YOLOv10 [3]처럼 재학습으로 NMS를 제거하거나, HE-BiDet [4]처럼 전용 가속기 안에서 조기 필터링을 수행한다. 객체 크기 문제는 주로 아키텍처·학습 단계에서 다루어지며, 적응형 임계값 연구 [5]도 학습 시점의 적응에 초점을 둔다. 본 연구는 재학습이나 전용 하드웨어 없이 표준 CPU의 후처리 단계에서 세 문제를 함께 다룬다는 점에서 구별된다.

III. 제안 기법

대상 검출기는 COCO [6]로 사전학습된 YOLOv8n [7]을 per-tensor INT8로 양자화한 모델이며, 416×416 입력에서 $N = 3,549$ 개의 격자 예측점(이하 앵커)을 출력한다. 표준 후처리(기준선 A)는 모든 앵커의 좌표를 역양자화·디코딩한 뒤 단일 임계값으로 거르고 NMS를 적용한다. 제안 기법은 모두 하나의 단일 패스 후처리 루프 안에서 플래그 조합으로 동작하며, 각 기법의 개입 지점은 그림 1과 같다.

양자화 인식 보정(B). 양자화 오차는 객체 크기에 따라 다르게 나타나므로, 크기 그룹 g (소형·중형·대형)와 좌표 (x, y, w, h) 마다 역양자화 좌표를 FP32 모델의 좌표로 되돌리는 선형 보정을 적용한다. 역양자화와 보정을 하나로 합치면 원시 INT8 값 q 에 직접 적용되는 식 (1)이 되며, m 과 b 는 보정 계수에 양자화 스케일과 영점을 미리 곱해 둔 융합 계수이다.

$$c_k = m_{g,k}q_k + b_{g,k} \quad (1)$$

따라서 보정은 앵커당 좌표 4개에 대한 곱·합 4회로 추가 비용 없이 수행된다. 직접 촬영한 225장의 표본 10,479개로 적합한 12개 회귀의 결정계수는 0.96–1.00이었고, 보정은 주로 y 와 h 채널에 집중되었다

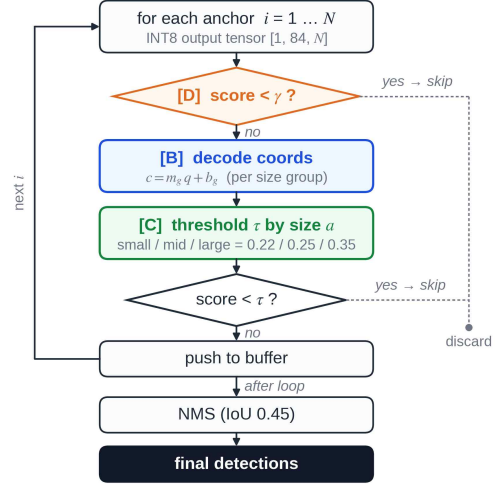


그림 1. 제안 기법의 후처리 루프. 주황·파랑·초록 상자는 각각 D·B·C가 개입하는 지점이다.

크기 인식 신뢰도 임계값(C). 소형 객체는 점수가 낮게 형성되므로, 정규화 면적 $a = w \cdot h$ 로 그룹을 정하고($a < 0.02$ 는 소형, $a \geq 0.10$ 는 대형) 소형·중형·대형에 각각 0.22, 0.25, 0.35의 임계값을 적용한다. 소형 임계값은 0.21–0.23 구간의 성능 안정 영역에서 선정하였다.

캐시 인식 선택적 디코딩(D). 대부분의 앵커는 대상 클래스 점수가 낮아 결국 폐기된다. D는 연속 배치된 대상 클래스 점수 채널만 먼저 순차 스캔하고, 점수가 사전 임계값 γ 이상인 앵커만 좌표를 읽어 디코딩한다. γ 를 그룹 임계값의 최소값(0.22)으로 두면 최종 검출될 수 있는 앵커는 모두 γ 를 넘으므로, D는 검출 결과를 바꾸지 않고 비용만 줄인다. 기준선 A와 D의 후처리 시간을 각각 T_A , T_D , 전체 앵커 수를 N , γ 를 넘은 앵커 수를 S , 앵커당 디코딩 비용을 c 라 하면 절감 시간은 식 (2)와 같다.

$$T_A - T_D \approx (N - S)c \quad (2)$$

통합 기법(E). C는 단독으로는 크기를 알기 위해 모든 앵커를 디코딩해야 해 비용이 늘지만, D와 결합하면 크기 판정이 γ 를 통과한 소수 앵커에서만 수행되어 추가 비용이 사라진다. 따라서 E는 B·C의 정확도 향상을 누적하면서 D 수준의 비용을 유지한다.

IV. 실험 결과

Raspberry Pi 4(Cortex-A72, 1.8GHz 고정)에서 TensorFlow Lite XNNPACK(4 threads)으로 측정하였다. 평가 데이터는 직접 촬영·주석한 225장(person 1,904개)이며, 워밍업 10회 후 30회 계측을 5회 반복한 평균을 보고한다. 모든 방법은 동일한 INT8 추론 출력(약 61ms)을 공유한다.

표 1. 기법별 후처리 시간과 정확도

기법	후처리(ms)	디코딩 앵커	mAP@0.5	mAP@ 0.5:0.95
A	0.3786	3,549	0.6203	0.4279
B	0.3565	3,549	0.6314	0.4323
C	0.4643	3,549	0.6298	0.4322
D	0.1316	48.0	0.6203	0.4279
C+D	0.1338	49.9	0.6298	0.4322
E	0.1324	49.9	0.6407	0.4368

표 1에서 E는 A와 총 지연시간 차이가 0.4%로 같은 16.3 FPS를 유지하면서 mAP@0.5를 +0.0204, mAP@0.5:0.95를 +0.0089 높였다. B(+0.0111)와 C(+0.0095)의 이득은 가법적이어서, 합으로 예측한 0.6409가 실측 0.6407과 거의 일치하였다. B는 역양자화 비용을 흡수해 후처리가 오히려 5.8% 줄었고, 개선은 소형 객체에 집중되었다(소형 AP@0.5 +0.0056). C는 단독으로 소형 객체의 재현율을 0.3827에서 0.4061로(+2.34%p) 높였으나, 모든 앵커를 디코딩해야 해 후처리 시간이 22.6% 늘었다. 반면 C+D에서는 C와 같은 정확도를 유지하면서 후처리 시간을 C 대비 71% 줄였다.

D는 디코딩 앵커를 3,549개에서 평균 48개로 줄여 후처리를 0.3786ms에서 0.1316ms로 65% 단축하였고, 225장 전체에서 A와 검출 결과가 동일하였다. 이미지별 절감 시간은 식 (2)대로 S에 따라 선형으로 감소하였다($R^2 = 0.947$). 그림 2는 같은 장면에서 A가 1명, E가 3명을 검출한 예로, 신뢰도 0.23의 인물은 전역 임계값 0.25에는 못 미치지만 소형 임계값 0.22를 넘어 채택되었다.

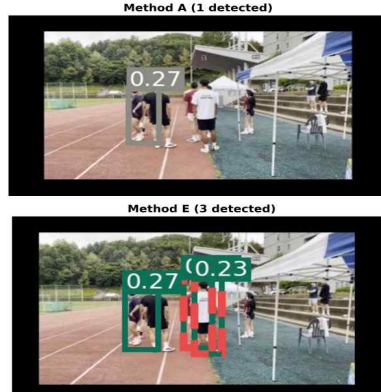


그림 2. 같은 장면에 대한 A와 E의 검출 결과 비교 (A: 1명, E: 3명)

V. 결 론

본 논문은 추론이 지배적인 ARM CPU 엡지 환경에서 후처리를 정확도와 메모리 효율의 최적화 지점으로 재정의하고, 양자화 인식 보정, 크기 인식 임계값, 캐시 인식 선택적 디코딩을 결합한 후처리를 제안하였다. Raspberry Pi 4 실측에서 제안 기법은 기준선과 같은 속도로 mAP@0.5를 +0.0204 높였고, 선택적 디코딩은 검출 결과를 바꾸지 않고 크기 인식 후처리의 추가 비용을 상쇄하였다. 평가는 단일 디바이스·단일 클래스·225장에 한정되므로, 향후 다양한 클래스와 디바이스, NPU 환경으로 검증을 확장할 예정이다.

References

- [1] B. Jacob, S. Kligys, B. Chen, M. Zhu, M. Tang, A. Howard, H. Adam, and D. Kalenichenko, "Quantization and training of neural networks for efficient integer-arithmetic-only inference," in Proceeding of the IEEE/CVF CVPR, Salt Lake City: UT, pp. 2704-2713, Jun. 2018.
- [2] Y. Ding, W. Feng, C. Chen, J. Guo, and X. Liu, "Reg-PTQ: Regression-specialized post-training quantization for fully quantized object detector," in Proceeding of the IEEE/CVF CVPR, Seattle: WA, pp. 16174-16184, Jun. 2024.
- [3] A. Wang, H. Chen, L. Liu, K. Chen, Z. Lin, J. Han, and G. Ding, "YOLOv10: Real-time end-to-end object detection," in Proceeding of the NeurIPS 2024, Vancouver: Canada, pp. 107984-108011, Dec. 2024.
- [4] D. Zhang, Z. Liang, R. Cen, Z. Yan, R. Wan, and D. Wang, "HE-BiDet: A hardware efficient binary neural network accelerator for object detection in SAR images," Micromachines, Vol. 16, No. 5, Art. no. 549, May 2025.
- [5] P. Kar, V. Chudasama, N. Onoe, and P. Wasnik, "Revisiting class imbalance for end-to-end semi-supervised object detection," in Proceeding of the IEEE/CVF CVPR Workshops, Vancouver: Canada, pp. 4570-4579, Jun. 2023.
- [6] T.-Y. Lin, M. Maire, S. Belongie, J. Hays, P. Perona, D. Ramanan, P. Dollár, and C. L. Zitnick, "Microsoft COCO: Common objects in context," in Proceeding of the ECCV, Zurich: Switzerland, pp. 740-755, Sep. 2014.
- [7] G. Jocher, A. Chaurasia, and J. Qiu. Ultralytics YOLOv8 [Internet]. Available : <https://github.com/ultralytics/ultralytics>.